Learning Objectives

Learners will be able to…

- **Describe the fundamental structure and properties of red-black trees, including their color-coding and balancing rules.**
- **Explain the role of node coloring and the rules for rotations in red-black trees to maintain balance and adhere to tree properties.**
- **Execute the process of inserting new nodes into red-black trees, ensuring maintenance of tree balance and color properties post-insertion.**
- **Implement the deletion process in red-black trees, addressing different scenarios of node removal and adjustments to preserve tree balance and coloring rules.**
- **Analyze the time and space complexity of red-black trees, particularly focusing on search, insertion, and deletion operations, to evaluate their efficiency for various applications.**

Introduction to Red-Black Trees

info

Red-Black Trees are a specialized form of binary search trees that maintain balance through node coloring and rotations, ensuring that operations such as insertion, deletion, and lookup can be performed in logarithmic time complexity.

Characteristics of Red-Black Trees

Red-Black Trees have unique properties that differentiate them from other binary search trees:

- **Node Color**: Each node is colored either *red* or *black*.
- **Root Property**: The *root* node is always *black*.
- **Leaf Property**: Every *NIL* leaf is *black*.

info

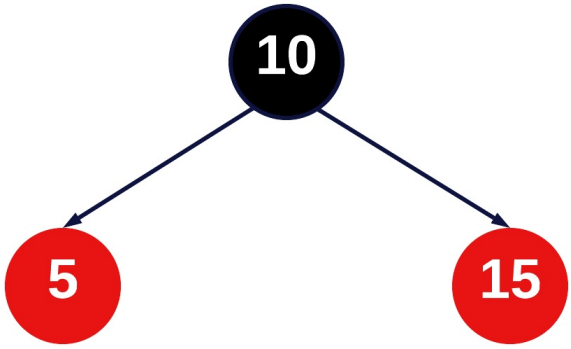
What Is a NIL Leaf?

A NIL leaf is a special node in a binary tree that does not have any children, meaning it has neither a left nor a right child. NIL nodes are used to represent the end of a tree branch and are essential in maintaining the structural integrity of the tree. Often, these NIL nodes are excluded from tree diagrams for simplicity. The term “NIL” is synonymous with “NULL,” which is commonly used to describe objects that have no value.

- **Red Node Property**: *Red* nodes *cannot* have red children (no two red nodes occur consecutively).
- **Black Depth Property**: All paths from a node to its leaf descendants contain the *same number* of black nodes.

Simple Example

Consider a simple red-black tree with the following nodes: 10 (Black), 5 (Red), 15 (Red).



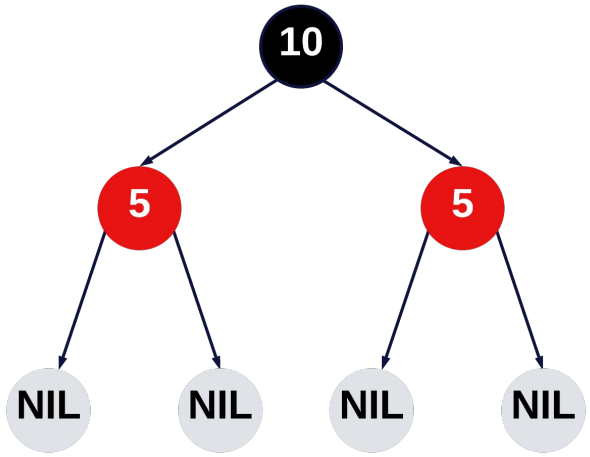
In this tree:

- The root (10) is black, satisfying the root property.
- The leaves (NIL nodes, not shown) are black by definition.
- The red nodes (5 and 15) do not have red children, maintaining the red node property.
- All paths from the root to the leaves have the same number of black nodes, satisfying the black depth property.

info

Diagram With NIL Nodes

As mentioned previously, NIL nodes are often excluded from tree diagrams for simplicity. Here is the same diagram with NIL nodes:



You can assume that there are NIL nodes at the end of all red-black trees.

Balancing the Tree

To maintain these properties, especially after insertions or deletions, red-black trees use rotations and recoloring. If a red node is inserted, for instance, and it causes two red nodes to be consecutive, the tree will perform necessary rotations and recolor certain nodes to restore balance. We'll cover these techniques a little later on.

Real-World Applications

Red-black trees are widely used in computer science. They form the backbone of many high-performance in-memory databases, help manage the blocks of memory in operating systems, and are used in various container libraries due to their ability to maintain balance and provide consistent performance. Thus, red-black trees are a critical data structure for efficient data management and retrieval, ensuring that as data grows, the performance does not degrade significantly.

Implementing a Red-Black Tree

Red-Black Nodes

Since a red-black tree is a self-balancing binary search tree, you are going to see some similarities to a node in a BST. We are also going to see some important differences as well. Create the `Node` class that stores a value and has left and right children. Notice how a node for a red-black tree also has attributes for the parent node and the color. Colors can be stored in a variety of ways. We are going to use integers; `1` means red and `0` means black. In the constructor, be sure to make every new node red.

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};
```

Create the Red-Black Tree

For now, we are going to build this tree similar to a BST. We will modify operations later on in the assignment. We are going to have the `root` attribute which represents the `root` of the tree. We are also going to make use of the default constructor to give `root` the value of `nullptr`. One of the properties of a `red-black` tree is that the `root` node is always black. Add a line of code that ensures that the `root` node is black.

```
class RedBlackTree {
private:
    Node *root;

    Node* insert(Node *root, Node *parent, int value) {
        if (root == nullptr) {
            Node *node = new Node(value);
            node->parent = parent;
            return node;
        }

        if (value < root->value) {
            root->left = insert(root->left, root, value);
        } else if (value > root->value) {
            root->right = insert(root->right, root, value);
        }

        return root; // duplicate value
    }

    void preOrder(Node *node) {
        if (node != nullptr) {
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, nullptr, value);
        root->color = 0; // root is always black
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }
};
```

Next, create the main method which instantiates a red-black tree. Then add the nodes `10`, `7`, `15`, and `12` to the tree. Finally, print the tree.

```
int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(7);
    tree.insert(15);
    tree.insert(12);

    // Print tree
    tree.preOrder();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};

class RedBlackTree {
private:
    Node *root;

    Node* insert(Node *root, Node *parent, int value) {
        if (root == nullptr) {
            Node *node = new Node(value);
            node->parent = parent;
            return node;
        }

        if (value < root->value) {
            root->left = insert(root->left, root, value);
        } else if (value > root->value) {
            root->right = insert(root->right, root, value);
        }

        return root; // duplicate value
    }

    void preOrder(Node *node) {
        if (node != nullptr) {
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
```

```
            preOrder(node->left);
            preOrder(node->right);
        }
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, nullptr, value);
        root->color = 0; // root is always black
    }

    void preOrder() {
        preOrder(root);
    }
};

int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(7);
    tree.insert(15);
    tree.insert(12);

    // Print tree
    tree.preOrder();

    return 0;
}
```

Remember, we force the root node to be black. All other nodes are created as red. We are not yet recoloring any nodes. You should see the following output:

```
10(Black) 7(Red) 15(Red) 12(Red)
```

Coloring and Rotations

Coloring and Rotations in Red-Black Trees

The Role of Coloring

In red-black trees, coloring is crucial for maintaining balance. The colors represent an additional piece of information that helps enforce a balanced tree structure. When a new node is inserted, it is always colored red. This potentially violates the red-black properties, which necessitates an adjustment through rotations and recoloring.

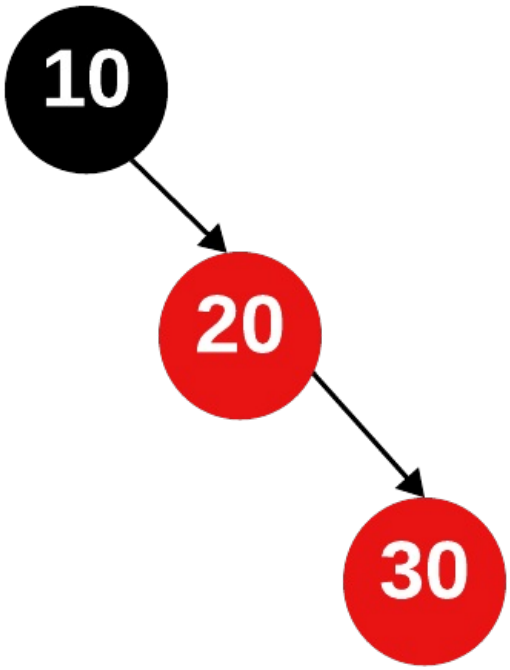
Rotations Explained

Rotations are operations that pivot nodes and edges in the tree to maintain the red-black properties. There are two primary types of rotations:

- **Left Rotation**: This rotation is performed when a right child is red, and its child is also red, or during deletion when re-balancing is required.
- **Right Rotation**: This rotation is applied when a left child is red, and its child is also red, or during deletion re-balancing.

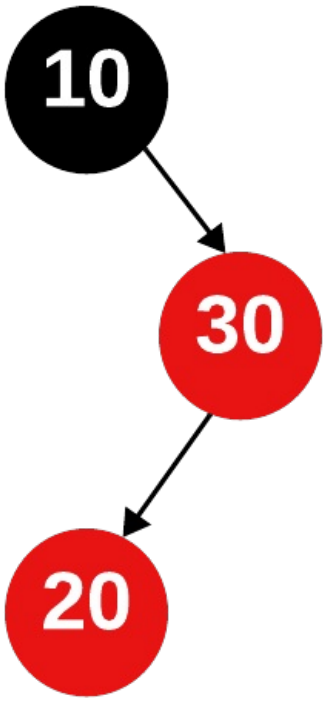
Example of Rotations

Consider a red-black tree where a red node `30R` is inserted as the right child of another red node `20R`, which is the right child of `10B`.



Red Black Tree Before Rotation

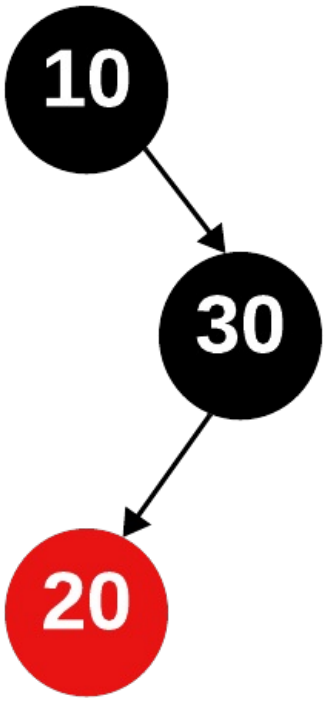
A left rotation on `20R` would yield:



After Left Rotation

Recoloring After Rotations

After a rotation, recoloring is often required to restore the red-black properties. In the above example, `10B` would remain black, `30R` would be recolored to black, and `20R` would remain red, maintaining the properties.



After Recoloring

Importance of Coloring and Rotations

Coloring and rotations are vital for self-balancing in red-black trees. They ensure that the tree remains approximately balanced after each insertion or deletion, which is critical for maintaining the tree's logarithmic height and, consequently, its operation efficiencies.

Understanding the mechanics of coloring and rotations is essential for anyone looking to implement red-black trees or understand their performance characteristics in depth.

Implementing Rotations

Left Rotation

Like AVL trees, red-black trees will use rotations to maintain balance. However, the logic behind the rotation is slightly different as red-black trees have a pointer for the parent node. We are going to start off with a simple rotation. Node `x` becomes the left subtree of node `y` and the left subtree of `y` becomes the right subtree of `x`.

```
private:
    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        return y;
    }
```

Next, we have to take into account the parent node. Since node `y` is now above node `x` in the `tree` hierarchy, the parent for node `x` is now the parent for node `y`. If the parent for node `y` is `nullptr`, that means node `y` is now the `root`. We then need to figure out if node `y` is the `left` or `right` child for the parent of node `x`. Finally, update the parent for node `x` to be node `y` and return node `y`.

```
private:
    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }
```

For testing purposes, we are going to make the public `makeRotation` method to force a rotation. We are only using this method for evaluation purposes. It will not be a part of our final code base.

```
public:
    void makeRotation() {
        leftRotate(root);
    }
```

Update the `main` method to add the nodes `10`, `20`, and `15`. Print the tree before the rotation, rotate the tree, and print it again.

```
int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(15);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "After rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};

class RedBlackTree {
private:
    Node *root;

    Node* insert(Node *root, Node *parent, int value) {
        if (root == nullptr) {
            Node *node = new Node(value);
            node->parent = parent;
            return node;
        }

        if (value < root->value) {
            root->left = insert(root->left, root, value);
        } else if (value > root->value) {
            root->right = insert(root->right, root, value);
        }

        return root; // duplicate value
    }

    void preOrder(Node *node) {
        if (node != nullptr) {
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
```

```
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, nullptr, value);
        root->color = 0; // root is always black
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }

    void makeRotation() {
        leftRotate(root);
    }
};

int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(15);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "After rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

You should see the output below. Ignore the colors for now since they have not been updated as a part of the rotation process:

```
Before rotation: 10(Black) 20(Red) 15(Red)
After rotation: 20(Red) 10(Black) 15(Red)
```

Right Rotation

A right rotation is going to be the mirror opposite of a left rotation. Node `x` becomes the right subtree of node `y`, and the right subtree of node `y` becomes the left subtree of node `x`. Then update the pointers for the parent nodes and return node `y`.

```
private:
    Node* rightRotate(Node* x) {
        Node* y = x->left;
        Node* newSubtree = y->right;

        y->right = x;
        x->left = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->right) {
            x->parent->right = y;
        } else {
            x->parent->left = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }
```

Update the `makeRotation` method to perform a right rotation instead of a left.

```
public:
    void makeRotation() {
        rightRotate(root);
    }
```

Then change the main method so that the nodes `20`, `10`, and `15` are added to the tree. Just as before print before and after the rotation to verify it is working as intended.

```
int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(20);
    tree.insert(10);
    tree.insert(15);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "After rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};

class RedBlackTree {
private:
    Node *root;

    Node* insert(Node *root, Node *parent, int value) {
        if (root == nullptr) {
            Node *node = new Node(value);
            node->parent = parent;
            return node;
        }

        if (value < root->value) {
            root->left = insert(root->left, root, value);
        } else if (value > root->value) {
            root->right = insert(root->right, root, value);
        }

        return root; // duplicate value
    }

    void preOrder(Node *node) {
        if (node != nullptr) {
```

```
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

    Node* rightRotate(Node* x) {
        Node* y = x->left;
        Node* newSubtree = y->right;

        y->right = x;
        x->left = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->right) {
            x->parent->right = y;
        } else {
            x->parent->left = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, nullptr, value);
        root->color = 0; // root is always black
```

```
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }

    void makeRotation() {
        rightRotate(root);
    }
};

int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(20);
    tree.insert(10);
    tree.insert(15);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "After rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

You should see the output below. Ignore the colors for now since they have not been updated as a part of the rotation process:

```
Before rotation: 20(Black) 10(Red) 15(Red)
After rotation: 10(Red) 20(Black) 15(Red)
```

Insertion in Red-Black Trees

Inserting Nodes

Overview of Insertion

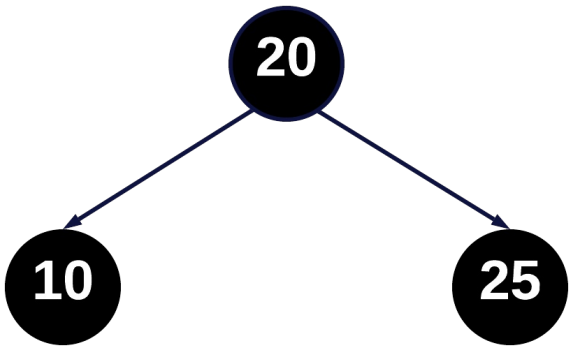
Insertion in red-black trees involves adding a new node to the tree and then adjusting the tree to maintain the red-black properties. The new node is always inserted as a red node.

Insertion Process

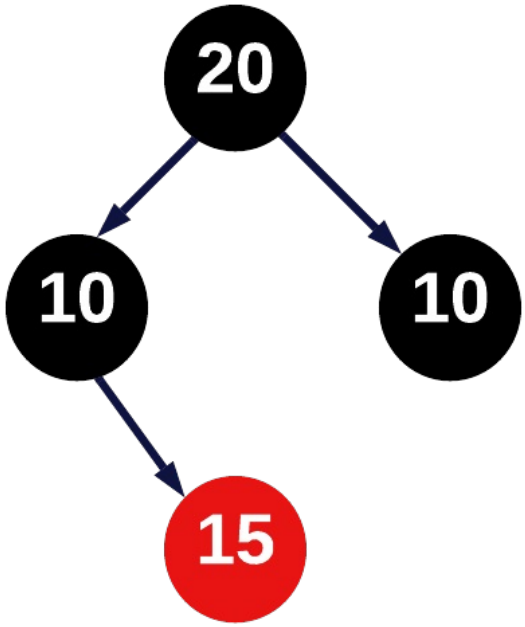
1. **Insert the Node**: Place the new node in the correct location in the tree, following the binary search tree insertion rules.
2. **Color the Node Red**: Color the new node red to maintain the red-black tree properties.
3. **Adjust the Tree**: If the parent of the new node is black, no further action is needed. If the parent is red, further adjustments are required to fix potential violations.

Example of Insertion

Consider inserting `15R` into the following red-black tree:



Before Insertion



After Insertion

Fixing Violations

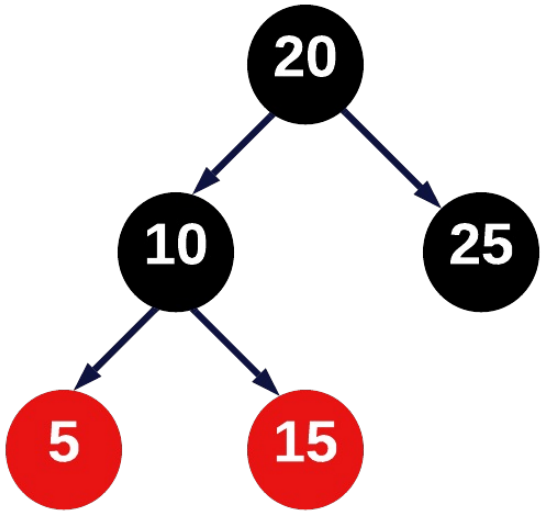
If the new red node's parent is also red, this violates the red-black properties. The tree must be adjusted through rotations and recoloring. The specific actions depend on the uncle's color (the sibling of the parent node).

Case Analysis

- **Case 1 (Uncle is Red)**: Recolor the parent and uncle to black, and the grandparent to red. Then, re-evaluate from the grandparent.
- **Case 2 (Uncle is Black, Triangle Formation)**: Perform a rotation on the parent to transform it into a line formation.
- **Case 3 (Uncle is Black, Line Formation)**: Perform a rotation on the grandparent and recolor the parent and grandparent.

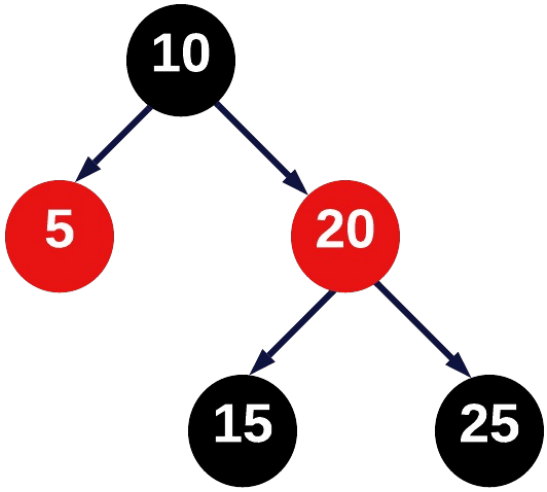
Example of Fixing Violations

Inserting `15R` when `10B` already has a red child `5R`:



Before Fixing Violations

After recoloring and rotations:



After Fixing Violations

Conclusion

The insertion process in red-black trees is meticulous but ensures that the tree remains balanced. It's a series of well-defined steps that, when followed correctly, maintain the red-black properties and the efficiency of the tree's operations.

Implementing an Insertion

info

Remove `makeRotation` Method

On a previous page, we created the `makeRotation` method to force a rotation on our tree. We are going incorporate this functionality in the `insert` method. Remove `makeRotation` as it is no longer needed.

```
// Remove this method from your code
void makeRotation() {
    root = rightRotate(root);
}
```

Inserting a Node

Up until now, we have been treating red-black nodes as `BST` node. That is, we have not been assigning a value to the parent attribute. This needs to change. In the public `insert` method, create a new node based on the given value. Update the value of root to the return value of the `private` insert method. Notice that the `private` method now takes three arguments. Then call `fixTree` to `fix` any violations after the insertion. Finally, set the color of the `root` to black.

```
    void insert(int value) {
        Node* newNode = new Node(value);
        root = insert(root, nullptr, newNode);
        fixTree(newNode); // Fix violations after insertion
        root->color = 0; // Ensure the root is always black
    }
```

We also need to update the `private` insert method. This method should now have three parameters: the current node, the parent of the current node, and the node to insert. When the current node is `nullptr`, set the parent attribute of the new `node` to the parent of the current node. Then return the new node.

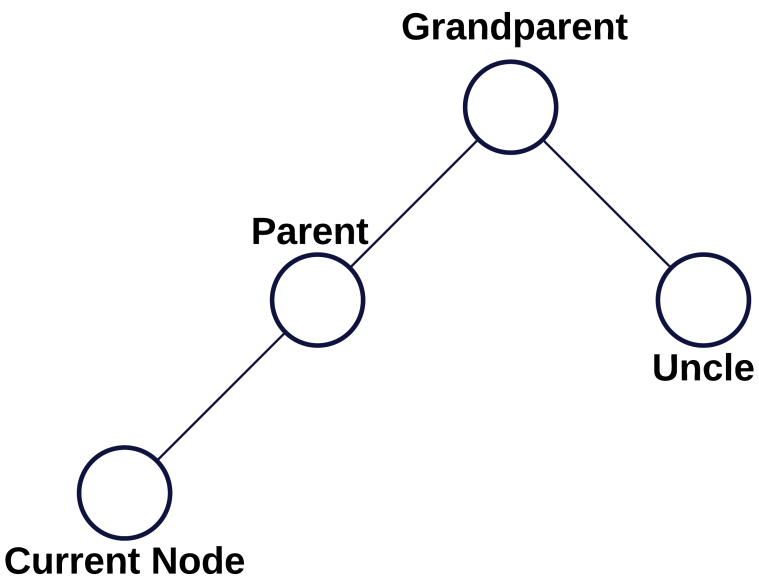
```
Node* insert(Node* node, Node* parent, Node* newNode) {
    if (node == nullptr) {
        newNode->parent = parent;
        return newNode;
    }

    if (newNode->value < node->value) {
        node->left = insert(node->left, node, newNode);
    } else if (newNode->value > node->value) {
        node->right = insert(node->right, node, newNode);
    }

    return node;
}
```

Fixing the Tree

For purposes of clarity, we are not going to create a recursive method, though that certainly can be done. Before we write any code, let's look at some vocabulary we are going to use to describe nodes in a tree. The parent of the current node is the node directly above in the tree hierarchy. The grandparent is the parent to the parent. The uncle is the sibling (left or right) of the parent. We will be using these terms in our `fixTree` method.



Begin by creating the `getUncle` helper method. This method takes a parent as a parameter and calculates the grandparent. If the parent is the left child of the grandparent, the uncle is the right child. Otherwise the uncle is the left child.

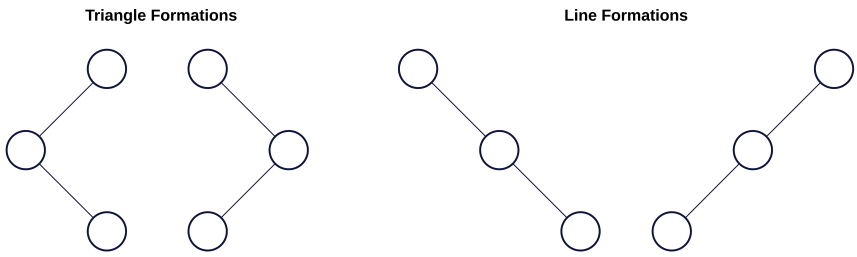
```
Node* getUncle(Node* parent) {
    Node* grandparent = parent->parent;

    if (grandparent->left == parent) {
        return grandparent->right;
    } else {
        return grandparent->left;
    }
}
```

We also need to remember the three different cases where there is a red-black violation and the tree needs to be fixed. Assume that the newly added node and its parent are both red.

- **Case 1 (Uncle is Red)**: Recolor the parent and uncle to black, and the grandparent to red. Then, re-evaluate from the grandparent.
- **Case 2 (Uncle is Black, Triangle Formation)**: Perform a rotation on the parent to transform it into a line formation.
- **Case 3 (Uncle is Black, Line Formation)**: Perform a rotation on the grandparent and recolor the parent and grandparent.

Cases 2 and 3 can exist as either the left or right subtree, so there are really five cases we need to address in our method. Terms like “triangle formation” and “line formation” may not be clear. Here is what these formations would look like. Remember, each formation has a mirror opposite.



Next, create the `fixTree` method. It takes a `Node` object as a parameter. Start by calculating the parent node. Use a conditional with three branches to check for Cases 1, 2, and 3 listed above. The core logic of fixing a tree goes in this conditional. The first branch checks for Case 1, a red uncle. The next branch checks to see if the parent is the left child of the grandparent, and the third branch is for when the parent is the right child of the grandparent.

```
    void fixTree(Node* node) {
        Node* parent = node->parent;

        if (parent == nullptr) {
            return; // Parent is null, we've reached the root,
        no changes made
        }

        if (parent->color == 0) {
            return; // Parent is black, no changes made
        }

        Node* grandparent = parent->parent;
        if (grandparent == nullptr) {
            parent->color = 0; // No grandparent means the
        parent is the root. Set to black.
            return;
        }

        Node* uncle = getUncle(parent);
        if (uncle != nullptr && uncle->color == 1) { // Case 1
        uncle is red
            parent->color = 0;
            grandparent->color = 1;
            uncle->color = 0;

            fixTree(grandparent); // Call recursively for
        grandparent
        } else if (parent == grandparent->left) { // Parent is
        left child of grandparent
            if (node == parent->right) { // Case 2 "<" triangle
        formation
                leftRotate(parent); // Rotate to "/" line
        formation
                parent = node;
            }

            rightRotate(grandparent); // Case 3 "/" line
        formation

            parent->color = 0;
            grandparent->color = 1;
        } else { // Parent is right child of grandparent
            if (node == parent->left) { // Case 2 ">" triangle
        formation
                rightRotate(parent); // Rotate to "\" line
        formation
                parent = node;
            }

            leftRotate(grandparent); // Case 3 "\" line
        formation

            parent->color = 0;
            grandparent->color = 1;
        }
    }
```

We can test our ability to properly insert, balance, and recolor a red-black tree by updating the main method. Add the nodes `45`, `23`, `13`, `50`, `79`, and `67` to the tree. Then print it.

```
int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(45);
    tree.insert(23);
    tree.insert(13);
    tree.insert(50);
    tree.insert(79);
    tree.insert(67);

    // Print tree
    tree.preOrder();

    return 0;
}
```

▼ Code

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};

class RedBlackTree {

private:
    Node* root;

Node* insert(Node* node, Node* parent, Node* newNode) {
    if (node == nullptr) {
        newNode->parent = parent;
        return newNode;
    }

    if (newNode->value < node->value) {
        node->left = insert(node->left, node, newNode);
    } else if (newNode->value > node->value) {
        node->right = insert(node->right, node, newNode);
    }

    return node;
}
```

```
void preOrder(Node* node) {
    if (node != nullptr) {
        std::string color = node->color == 1 ? "Red" :
    "Black";
        std::cout << node->value << "(" << color << ") ";
        preOrder(node->left);
        preOrder(node->right);
    }
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* newSubtree = y->left;

    y->left = x;
    x->right = newSubtree;

    y->parent = x->parent;

    if (x->parent == nullptr) {
        root = y;
    } else if (x == x->parent->left) {
        x->parent->left = y;
    } else {
        x->parent->right = y;
    }

    x->parent = y;

    return y; // Return the new root of the rotated subtree
}

Node* rightRotate(Node* x) {
    Node* y = x->left;
    Node* newSubtree = y->right;

    y->right = x;
    x->left = newSubtree;

    y->parent = x->parent;

    if (x->parent == nullptr) {
        root = y;
    } else if (x == x->parent->right) {
        x->parent->right = y;
    } else {
        x->parent->left = y;
    }

    x->parent = y;

    return y; // Return the new root of the rotated subtree
}

void fixTree(Node* node) {
    Node* parent = node->parent;
```

```
    if (parent == nullptr) {
        return; // Parent is null, we've reached the root,
    no changes made
    }

    if (parent->color == 0) {
        return; // Parent is black, no changes made
    }

    Node* grandparent = parent->parent;
    if (grandparent == nullptr) {
        parent->color = 0; // No grandparent means the
    parent is the root. Set to black.
        return;
    }

    Node* uncle = getUncle(parent);
    if (uncle != nullptr && uncle->color == 1) { // Case 1
    uncle is red
        parent->color = 0;
        grandparent->color = 1;
        uncle->color = 0;

        fixTree(grandparent); // Call recursively for
    grandparent
    } else if (parent == grandparent->left) { // Parent is
    left child of grandparent
        if (node == parent->right) { // Case 2 "<" triangle
    formation
            leftRotate(parent); // Rotate to "/" line
    formation
            parent = node;
        }

        rightRotate(grandparent); // Case 3 "/" line
    formation

        parent->color = 0;
        grandparent->color = 1;
    } else { // Parent is right child of grandparent
        if (node == parent->left) { // Case 2 ">" triangle
    formation
            rightRotate(parent); // Rotate to "\" line
    formation
            parent = node;
        }

        leftRotate(grandparent); // Case 3 "\" line
    formation

        parent->color = 0;
        grandparent->color = 1;
    }
}

Node* getUncle(Node* parent) {
    Node* grandparent = parent->parent;

    if (grandparent->left == parent) {
        return grandparent->right;
```

```
        } else {
            return grandparent->left;
        }
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        Node* newNode = new Node(value);
        root = insert(root, nullptr, newNode);
        fixTree(newNode); // Fix violations after insertion
        root->color = 0; // Ensure the root is always black
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }
};

int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(45);
    tree.insert(23);
    tree.insert(13);
    tree.insert(50);
    tree.insert(79);
    tree.insert(67);

    // Print tree
    tree.preOrder();

    return 0;
}
```

You should see the following output:

```
23(Black) 13(Black) 50(Red) 45(Black) 79(Black) 67(Red)
```

Deletion in Red-Black Trees

Deleting Nodes

Overview of Deletion

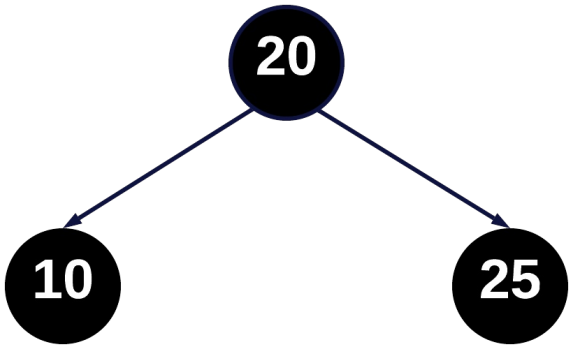
Deletion in red-black trees is a process that removes a node and then makes adjustments to preserve the red-black properties. It is more complex than insertion due to the need to replace the deleted node and fix potential violations.

Deletion Process

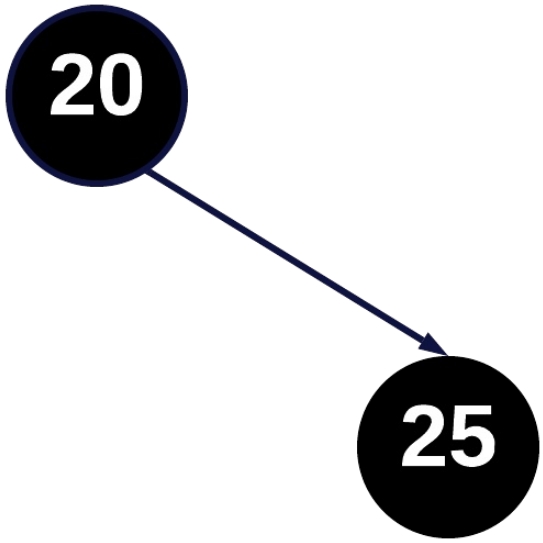
1. **Find the Node**: Locate the node to be deleted using binary search tree rules.
2. **Remove the Node**: If the node has two children, swap it with its in-order successor (or predecessor) which will have at most one child, then remove it.
3. **Adjust the Tree**: If the removed node was red, there's no violation. If it was black, the tree needs adjustments to fix the black height property.

Example of Deletion

Consider deleting `10B` from the following red-black tree:



Before Deletion



After Deletion

Fixing Violations

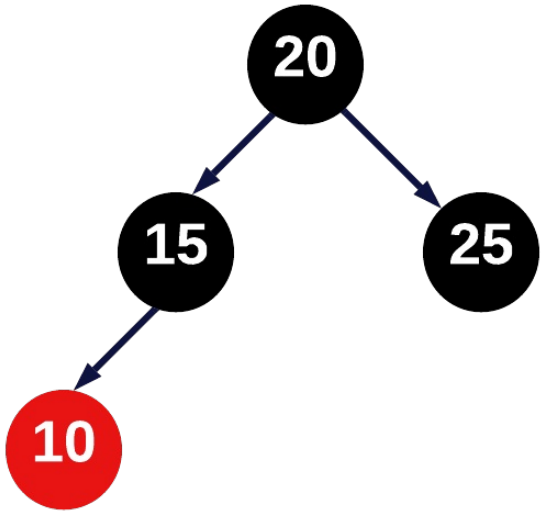
If a black node is removed, it creates a “double black” issue that needs to be resolved through tree rotations and recoloring to redistribute the black height across the tree.

Case Analysis

- **Case 1 (Sibling is Red)**: Perform a rotation to switch the sibling and parent colors, converting into one of the next cases.
- **Case 2 (Sibling and Sibling’s Children are Black)**: Repaint the sibling red and move the double black up to the parent.
- **Case 3 (Sibling is Black with a Red Child)**: Perform rotations and recoloring to remove the double black and balance the tree.

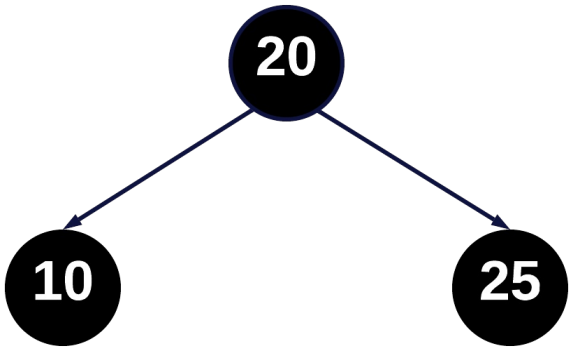
Example of Fixing Violations

Deleting `15B` when it has a red child `10R`:



Before Fixing Violations

After fixing the double black issue:



After Fixing Violations

Conclusion

Deletion in red-black trees requires careful consideration of the tree's properties. The process involves not only removing a node but also ensuring that the tree's balance is maintained through a series of well-defined adjustments.

Implementing Deletion

Helper Methods

We are going to start the deletion process by creating some helper methods that will be used. The first method is `search`. We need to locate the node that is to be deleted. This is a recursive method which takes the current node in the tree and the search value. Recurse down the left subtree if the search value is less than the value of the current node. Otherwise, recurse down the right subtree. If the current node is nullptr or the search target is equal to the current node's value, return the current node.

```
Node* search(Node* node, int value) {
    if (node == nullptr || node->value == value) {
        return node;
    }

    if (value < node->value) {
        return search(node->left, value);
    } else {
        return search(node->right, value);
    }
}
```

Next, we need the `findMin` method that will find the node with the minimum value in a subtree. This method takes the root of a subtree. Iterate over the left children until you find a leaf node. Return the leaf node.

```
Node* findMin(Node* node) {
    Node* current = node;

    while (current->left != nullptr)
        current = current->left;

    return current;
}
```

The `transplant` method moves subtrees within the tree. If the parent of the node to delete is `nullptr`, that means the node to delete is the root of the tree. In this case, have root point to the node to be moved. If the node to delete is the `left` child, have the left attribute of the parent node now point to the node to be moved. If the node to delete is the right child, have the right attribute of the parent node now point to the node to be moved. Finally, if the node to be moved is not nullptr, have the parent of the node

to be deleted now point to the node to be moved. Notice that the `transplant` method does not update the child nodes of nodeToMove. This will be handled by the `deleteNode` method.

```
    void transplant(Node* nodeToDelete, Node* nodeToMove) {
        if (nodeToDelete->parent == nullptr) {
            root = nodeToMove;
        } else if (nodeToDelete == nodeToDelete->parent->left) {
            nodeToDelete->parent->left = nodeToMove;
        } else {
            nodeToDelete->parent->right = nodeToMove;
        }

        if (nodeToMove != nullptr) {
            nodeToMove->parent = nodeToDelete->parent;
        }
    }
```

Deleting a Node

To delete a node, start with the public-facing `deleteNode` method. This method takes the value to be deleted. First search the tree to see if the node is even there. If yes, pass the node to delete to the private deleteNode method.

```
    void deleteNode(int value) {
        Node* nodeToDelete = search(root, value); // Find the
        node to delete
        if (nodeToDelete != nullptr) {
            deleteNode(nodeToDelete);
        }
    }
```

Just like with BSTs, deletion is rather simple when there is only one child, but becomes more complicated if the node to delete has two children. Create temporary variables for the replacement node and the node to be fixed. We also need to note the original color of the node to be deleted. Set up a conditional that has a branch for when the node to delete has a right child, for when the node to delete has a left child, and when the node to delete has two children.

```
    void deleteNode(Node* node) {
        Node* replacement = nullptr;
        Node* toFix = nullptr;
        int originalColor = node->color;

        if (node->left == nullptr) { // node to delete has right
        child

        } else if (node->right == nullptr) { // node to delete
        has right child

        } else { // node to delete has two children

        }

    }
```

For brevity's sake, we are not going to show the entirety of the `deleteNode` method as we add more logic to it. Please note where the new lines of code are being added. If the node to delete has a right child, the replacement node is the right child. The replacement node is also the node to be fixed. Call the `transplant` method and pass it the node to be deleted and its right child.

```
        if (node->left == nullptr) { // node to delete has right
        child
            replacement = node->right;
            toFix = replacement;
            transplant(node, node->right);
        }
```

If the node to delete has a left child, the replacement node is the right child. The replacement node is also the node to be fixed. Call the `transplant` method and pass it the node to be deleted and its left child.

```
        } else if (node->right == nullptr) { // node to delete
        has right child
            replacement = node->left;
            toFix = replacement;
            transplant(node, node->left);
        }
```

If the node to delete has two children, then we have to take a few more steps. First calculate the successor by finding the smallest node in the right subtree. Change the originalColor variable to the color of the successor. The replacement node now becomes the right child of the successor. The node to fix is also the replacement node, as long as the replacement node is not `nullptr`. Otherwise, the node to fix becomes the parent of the successor.

```
    } else { // node to delete has two children
        Node* successor = findMin(node->right);
        originalColor = successor->color;
        replacement = successor->right;
        toFix = replacement != nullptr ? replacement :
    successor->parent;

    }
```

Now that we have the successor node, there are two possible cases. One, the successor is the right child of the node being deleted. As long as the replacement node is not `nullptr` have the parent of the parent node point to the successor. Two, if the successor is not the right child, call `transplant` and pass it the successor and the successor's right subtree. Then have the right child of the successor point to the right subtree of the node to be deleted. As long as the right child of the successor is not `nullptr`, have the parent of the right child point to the successor.

```
    } else { // node to delete has two children
        Node* successor = findMin(node->right);
        originalColor = successor->color;
        replacement = successor->right;
        toFix = replacement != nullptr ? replacement :
    successor->parent;

        if (successor->parent == node) {
            if (replacement != nullptr) {
                replacement->parent = successor;
            }
        } else {
            transplant(successor, successor->right);
            successor->right = node->right;
            if (successor->right != nullptr) {
                successor->right->parent = successor;
            }
        }

    }
```

To finish the `deleteNode` method, swap the node and the successor with the `transplant` method. Have the left subtree of the successor point to the left subtree of the node to be deleted. Then have the parent of the left subtree point to the successor. The color of the successor should now be the color of the node to be deleted. Finally, if the original color is black, we need to call the `fixDelete` method to maintain the properties of a red-black tree.

```
        } else {// node to delete has two children
            Node* successor = findMin(node->right);
            originalColor = successor->color;
            replacement = successor->right;
            toFix = replacement != nullptr ? replacement :
        successor->parent;

            if (successor->parent == node) {
                if (replacement != nullptr) {
                    replacement->parent = successor;
                }
            } else {
                transplant(successor, successor->right);
                successor->right = node->right;
                if (successor->right != nullptr) {
                    successor->right->parent = successor;
                }
            }

            transplant(node, successor);
            successor->left = node->left;
            successor->left->parent = successor;
            successor->color = node->color;
        }

        if (originalColor == 0) {
            fixDelete(toFix);
        }
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};


class RedBlackTree {
private:
    Node* root;

Node* insert(Node* node, Node* parent, Node* newNode) {
    if (node == nullptr) {
        newNode->parent = parent;
        return newNode;
    }
```

```
    if (newNode->value < node->value) {
        node->left = insert(node->left, node, newNode);
    } else if (newNode->value > node->value) {
        node->right = insert(node->right, node, newNode);
    }

    return node;
}

    void preOrder(Node* node) {
        if (node != nullptr) {
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

    Node* rightRotate(Node* x) {
        Node* y = x->left;
        Node* newSubtree = y->right;

        y->right = x;
        x->left = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->right) {
            x->parent->right = y;
        } else {
            x->parent->left = y;
        }
```

```
    x->parent = y;

    return y; // Return the new root of the rotated subtree
}

void fixTree(Node* node) {
    Node* parent = node->parent;

    if (parent == nullptr) {
        return; // Parent is null, we've reached the root,
    no changes made
    }

    if (parent->color == 0) {
        return; // Parent is black, no changes made
    }

    Node* grandparent = parent->parent;
    if (grandparent == nullptr) {
        parent->color = 0; // No grandparent means the
    parent is the root. Set to black.
        return;
    }

    Node* uncle = getUncle(parent);
    if (uncle != nullptr && uncle->color == 1) { // Case 1
    uncle is red
        parent->color = 0;
        grandparent->color = 1;
        uncle->color = 0;

        fixTree(grandparent); // Call recursively for
    grandparent
    } else if (parent == grandparent->left) { // Parent is
    left child of grandparent
        if (node == parent->right) { // Case 2 "<" triangle
    formation
            leftRotate(parent); // Rotate to "/" line
    formation
            parent = node;
        }

        rightRotate(grandparent); // Case 3 "/" line
    formation

        parent->color = 0;
        grandparent->color = 1;
    } else { // Parent is right child of grandparent
        if (node == parent->left) { // Case 2 ">" triangle
    formation
            rightRotate(parent); // Rotate to "\" line
    formation
            parent = node;
        }

        leftRotate(grandparent); // Case 3 "\" line
    formation

        parent->color = 0;
        grandparent->color = 1;
```

```
    }
}

Node* getUncle(Node* parent) {
    Node* grandparent = parent->parent;

    if (grandparent->left == parent) {
        return grandparent->right;
    } else {
        return grandparent->left;
    }
}
  Node* search(Node* node, int value) {
        if (node == nullptr || node->value == value) {
            return node;
        }

        if (value < node->value) {
            return search(node->left, value);
        } else {
            return search(node->right, value);
        }
    }

    Node* findMin(Node* node) {
        Node* current = node;

        while (current->left != nullptr)
            current = current->left;

        return current;
    }

    void transplant(Node* u, Node* v) {
        if (u->parent == nullptr) {
            root = v;
        } else if (u == u->parent->left) {
            u->parent->left = v;
        } else {
            u->parent->right = v;
        }

        if (v != nullptr) {
            v->parent = u->parent;
        }
    }
     void deleteNode(Node* node) {
        Node* replacement = nullptr;
        Node* toFix = nullptr;
        int originalColor = node->color;

        if (node->left == nullptr) { // node to delete has
    right child
            replacement = node->right;
            toFix = replacement;
            transplant(node, node->right);
        } else if (node->right == nullptr) { // node to
    delete has left child
```

```
                replacement = node->left;
                toFix = replacement;
                transplant(node, node->left);
            } else { // node to delete has two children
                Node* successor = findMin(node->right);
                originalColor = successor->color;
                replacement = successor->right;
                toFix = replacement != nullptr ? replacement :
        successor->parent;

                if (successor->parent == node) {
                    if (replacement != nullptr) {
                        replacement->parent = successor;
                    }
                } else {
                    transplant(successor, successor->right);
                    successor->right = node->right;
                    if (successor->right != nullptr) {
                        successor->right->parent = successor;
                    }
                }

                transplant(node, successor);
                successor->left = node->left;
                successor->left->parent = successor;
                successor->color = node->color;
            }

            if (originalColor == 0) {
                fixDelete(toFix);
            }
        }

        void fixDelete(Node* node) {
            // Implementation of fixDelete will be added later.
        }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        Node* newNode = new Node(value);
        root = insert(root, nullptr, newNode);
        fixTree(newNode); // Fix violations after insertion
        root->color = 0; // Ensure the root is always black
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }

    void makeRotation() {
         rightRotate(root);
    }

    void deleteNode(int value) {
```

```
        Node* nodeToDelete = search(root, value); // Find the
        node to delete
        if (nodeToDelete != nullptr) {
            deleteNode(nodeToDelete);
        }
    }
};
```

We are not yet ready to run the code, as we still need to implement the `fixDelete` method. We'll do this on the next page.

Fixing Deletions

Updating the Tree After a Deletion

After deleting the node from the tree, we need to go back and fix any violations in the red-black tree. This is why we had the `toFix` variable. It tells the `fixDelete` method where to begin. Use a while loop to iterate up the tree starting at the `toFix` node. The node to fix can either be the left child or the right child.

```
    void fixDelete(Node* node) {
        while (node != root && node->color == 0) {
            Node* sibling;
            if (node == node->parent->left) { // node is the
        left child

            } else { // node is the right child


            }
        } // end while loop
    }
```

If the node to fix is the left child, find its sibling. If the sibling is red, set the sibling to black, the parent of the node to fix becomes red. Then perform a left rotate around the node to fix's parent. Update the sibling.

```
                if (node == node->parent->left) { // node is the
        left child
                    sibling = node->parent->right;
                    if (sibling->color == 1) {
                        sibling->color = 0;
                        node->parent->color = 1;
                        leftRotate(node->parent);
                        sibling = node->parent->right;
                    }
                }
```

If the children of the sibling are both black, make the sibling red. Then move up one level in the tree.

```
            if (node == node->parent->left) { // node is the
        left child
                sibling = node->parent->right;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    leftRotate(node->parent);
                    sibling = node->parent->right;
                }
                if (sibling->left->color == 0 && sibling-
        >right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                }
            }
```

If both children of the sibling are not black, check to see if the right child is black. If so, set the left child to black and the sibling to red. Do a right rotation around the sibling. Update the sibling.

```
            if (node == node->parent->left) { // node is the
        left child
                sibling = node->parent->right;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    leftRotate(node->parent);
                    sibling = node->parent->right;
                }
                if (sibling->left->color == 0 && sibling-
        >right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                } else {
                    if (sibling->right->color == 0) {
                        sibling->left->color = 0;
                        sibling->color = 1;
                        rightRotate(sibling);
                        sibling = node->parent->right;
                    }
                }
            }
```

Next, update the sibling color to match the parent color. Then change the parent color to black. The right child of the sibling also needs to be black. Do a left rotate around the node to fix's parent, and set the node to root.

```
            if (node == node->parent->left) { // node is the
        left child
                sibling = node->parent->right;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    leftRotate(node->parent);
                    sibling = node->parent->right;
                }
                if (sibling->left->color == 0 && sibling-
        >right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                } else {
                    if (sibling->right->color == 0) {
                        sibling->left->color = 0;
                        sibling->color = 1;
                        rightRotate(sibling);
                        sibling = node->parent->right;
                    }
                    sibling->color = node->parent->color;
                    node->parent->color = 0;
                    sibling->right->color = 0;
                    leftRotate(node->parent);
                    node = root;
                }
            }
```

If the node to fix is the right child, find its sibling. If the sibling is red, set the sibling to black and the parent to red. Do a right rotation around the node to fix's parent and update the sibling.

```
            } else { // node is the right child
                sibling = node->parent->left;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    rightRotate(node->parent);
                    sibling = node->parent->left;
                }
            }
```

If the children of the sibling are both black, set the sibling to red and move up the tree.

```
            } else { // node is the right child
                sibling = node->parent->left;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    rightRotate(node->parent);
                    sibling = node->parent->left;
                }
                if (sibling->right->color == 0 && sibling-
>right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                }
            }
```

If the left child of the sibling is black, set the right child of the sibling to black as well. The sibling should then be red. Perform a left rotation around the sibling, and then update the sibling.

```
            } else { // node is the right child
                sibling = node->parent->left;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    rightRotate(node->parent);
                    sibling = node->parent->left;
                }
                if (sibling->right->color == 0 && sibling-
>right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                } else {
                    if (sibling->left->color == 0) {
                        sibling->right->color = 0;
                        sibling->color = 1;
                        leftRotate(sibling);
                        sibling = node->parent->left;
                    }
                }
            }
```

Next, set the sibling color to the color of the node to fix's parent. Then change the color of the node to fix to black. The left child of the sibling should also be black. Perform a right rotation around the node to fix's parent. Then set the node to root.

```
            } else { // node is the right child
                sibling = node->parent->left;
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    rightRotate(node->parent);
                    sibling = node->parent->left;
                }
                if (sibling->right->color == 0 && sibling-
>right->color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                } else {
                    if (sibling->left->color == 0) {
                        sibling->right->color = 0;
                        sibling->color = 1;
                        leftRotate(sibling);
                        sibling = node->parent->left;
                    }
                    sibling->color = node->parent->color;
                    node->parent->color = 0;
                    sibling->left->color = 0;
                    rightRotate(node->parent);
                    node = root;
                }
            }
```

Finally, if the node is not null, set it to black.

```
        } // end while loop
        if (node != nullptr) {
            node->color = 0;
        }
```

Update the `main` method so that the red-black tree has the nodes `62`, `33`, `99`, `57`, `75`, `129`, `67`, and `80`. Print the tree before the deletion. Then delete node `99` and print the tree again.

```
int main() {
    RedBlackTree tree;

    // Insert nodes
    tree.insert(62);
    tree.insert(33);
    tree.insert(99);
    tree.insert(57);
    tree.insert(75);
    tree.insert(129);
    tree.insert(67);
    tree.insert(80);

    // Print tree
    std::cout << "Before Deletion" << std::endl;
    tree.preOrder();
    std::cout << "\nAfter Deletion" << std::endl;
    tree.deleteNode(99);
    tree.preOrder();

    return 0;
}
```

▼ Code

Your code should look like this:

```
class Node {
public:
    int value;
    Node *parent, *left, *right;
    int color; // 0 for black, 1 for red

    Node(int v) : value(v), parent(nullptr), left(nullptr),
        right(nullptr), color(1) {
        // New nodes are always red
    }
};

class RedBlackTree {
private:
    Node* root;

Node* insert(Node* node, Node* parent, Node* newNode) {
    if (node == nullptr) {
        newNode->parent = parent;
        return newNode;
    }

    if (newNode->value < node->value) {
        node->left = insert(node->left, node, newNode);
    } else if (newNode->value > node->value) {
        node->right = insert(node->right, node, newNode);
    }
```

```
    return node;
}

    void preOrder(Node* node) {
        if (node != nullptr) {
            std::string color = node->color == 1 ? "Red" :
        "Black";
            std::cout << node->value << "(" << color << ") ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    Node* leftRotate(Node* x) {
        Node* y = x->right;
        Node* newSubtree = y->left;

        y->left = x;
        x->right = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

    Node* rightRotate(Node* x) {
        Node* y = x->left;
        Node* newSubtree = y->right;

        y->right = x;
        x->left = newSubtree;

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->right) {
            x->parent->right = y;
        } else {
            x->parent->left = y;
        }

        x->parent = y;

        return y; // Return the new root of the rotated subtree
    }

    void fixTree(Node* node) {
```

```
    Node* parent = node->parent;

    if (parent == nullptr) {
        return; // Parent is null, we've reached the root,
    no changes made
    }

    if (parent->color == 0) {
        return; // Parent is black, no changes made
    }

    Node* grandparent = parent->parent;
    if (grandparent == nullptr) {
        parent->color = 0; // No grandparent means the
    parent is the root. Set to black.
        return;
    }

    Node* uncle = getUncle(parent);
    if (uncle != nullptr && uncle->color == 1) { // Case 1
    uncle is red
        parent->color = 0;
        grandparent->color = 1;
        uncle->color = 0;

        fixTree(grandparent); // Call recursively for
    grandparent
    } else if (parent == grandparent->left) { // Parent is
    left child of grandparent
        if (node == parent->right) { // Case 2 "<" triangle
    formation
            leftRotate(parent); // Rotate to "/" line
    formation
            parent = node;
        }

        rightRotate(grandparent); // Case 3 "/" line
    formation

        parent->color = 0;
        grandparent->color = 1;
    } else { // Parent is right child of grandparent
        if (node == parent->left) { // Case 2 ">" triangle
    formation
            rightRotate(parent); // Rotate to "\" line
    formation
            parent = node;
        }

        leftRotate(grandparent); // Case 3 "\" line
    formation

        parent->color = 0;
        grandparent->color = 1;
    }
}

Node* getUncle(Node* parent) {
    Node* grandparent = parent->parent;
```

```
        if (grandparent->left == parent) {
            return grandparent->right;
        } else {
            return grandparent->left;
        }
    }
      Node* search(Node* node, int value) {
            if (node == nullptr || node->value == value) {
                return node;
            }

            if (value < node->value) {
                return search(node->left, value);
            } else {
                return search(node->right, value);
            }
        }

        Node* findMin(Node* node) {
            Node* current = node;

            while (current->left != nullptr)
                current = current->left;

            return current;
        }

        void transplant(Node* u, Node* v) {
            if (u->parent == nullptr) {
                root = v;
            } else if (u == u->parent->left) {
                u->parent->left = v;
            } else {
                u->parent->right = v;
            }

            if (v != nullptr) {
                v->parent = u->parent;
            }
        }
         void deleteNode(Node* node) {
            Node* replacement = nullptr;
            Node* toFix = nullptr;
            int originalColor = node->color;

            if (node->left == nullptr) { // node to delete has
        right child
                replacement = node->right;
                toFix = replacement;
                transplant(node, node->right);
            } else if (node->right == nullptr) { // node to
        delete has left child
                replacement = node->left;
                toFix = replacement;
                transplant(node, node->left);
            } else { // node to delete has two children
                Node* successor = findMin(node->right);
                originalColor = successor->color;
```

```
            replacement = successor->right;
            toFix = replacement != nullptr ? replacement :
        successor->parent;

            if (successor->parent == node) {
                if (replacement != nullptr) {
                    replacement->parent = successor;
                }
            } else {
                transplant(successor, successor->right);
                successor->right = node->right;
                if (successor->right != nullptr) {
                    successor->right->parent = successor;
                }
            }

            transplant(node, successor);
            successor->left = node->left;
            successor->left->parent = successor;
            successor->color = node->color;
        }

        if (originalColor == 0) {
            fixDelete(toFix);
        }
    }

void fixDelete(Node* node) {
    while (node != root && node->color == 0) {
        Node* sibling;
        if (node == node->parent->left) { // node is the
    left child
            sibling = node->parent->right;
            if (sibling->color == 1) {
                sibling->color = 0;
                node->parent->color = 1;
                leftRotate(node->parent);
                sibling = node->parent->right;
            }
            if (sibling->left->color == 0 && sibling->right-
    >color == 0) {
                sibling->color = 1;
                node = node->parent;
            } else {
                if (sibling->right->color == 0) {
                    sibling->left->color = 0;
                    sibling->color = 1;
                    rightRotate(sibling);
                    sibling = node->parent->right;
                }
                sibling->color = node->parent->color;
                node->parent->color = 0;
                sibling->right->color = 0;
                leftRotate(node->parent);
                node = root;
            }
        } else { // node is the right child
            sibling = node->parent->left;
```

```
                if (sibling->color == 1) {
                    sibling->color = 0;
                    node->parent->color = 1;
                    rightRotate(node->parent);
                    sibling = node->parent->left;
                }
                if (sibling->right->color == 0 && sibling->left-
        >color == 0) {
                    sibling->color = 1;
                    node = node->parent;
                } else {
                    if (sibling->left->color == 0) {
                        sibling->right->color = 0;
                        sibling->color = 1;
                        leftRotate(sibling);
                        sibling = node->parent->left;
                    }
                    sibling->color = node->parent->color;
                    node->parent->color = 0;
                    sibling->left->color = 0;
                    rightRotate(node->parent);
                    node = root;
                }
            }
        }
        if (node != nullptr) {
            node->color = 0;
        }
    }

public:
    RedBlackTree() : root(nullptr) {}

    void insert(int value) {
        Node* newNode = new Node(value);
        root = insert(root, nullptr, newNode);
        fixTree(newNode); // Fix violations after insertion
        root->color = 0; // Ensure the root is always black
    }

    void preOrder() {
        preOrder(root);
        std::cout << std::endl;
    }

    void makeRotation() {
         rightRotate(root);
    }

    void deleteNode(int value) {
        Node* nodeToDelete = search(root, value); // Find the
        node to delete
        if (nodeToDelete != nullptr) {
            deleteNode(nodeToDelete);
        }
    }
};
```

```
int main() {
    RedBlackTree tree;
    // Insert nodes
    tree.insert(62);
    tree.insert(33);
    tree.insert(99);
    tree.insert(57);
    tree.insert(75);
    tree.insert(129);
    tree.insert(67);
    tree.insert(80);

    // Print tree
    std::cout << "Before Deletion" << std::endl;
    tree.preOrder();
    std::cout << "\nAfter Deletion" << std::endl;
    tree.deleteNode(99);
    tree.preOrder();

    return 0;
}
```

You should see the following output:

```
Before Deletion
62(Black) 33(Black) 57(Red) 99(Red) 75(Black) 67(Red) 80(Red)
        129(Black)

After Deletion
62(Black) 33(Black) 57(Red) 129(Red) 75(Black) 67(Red) 80(Red)
```

Time and Space Complexity of Red-Black Trees

Analyzing Red-Black Trees

Time Complexity Overview

The time complexity of red-black trees is crucial for understanding their efficiency in various operations such as search, insertion, and deletion.

Search Operation

The search operation in a red-black tree is similar to that in a binary search tree (BST), with the added benefit of balanced height due to the red-black properties.

- **Best Case**: $O(\log n)$ when the tree is balanced.
- **Worst Case**: $O(\log n)$, which is also logarithmic because the tree is always balanced.

Insertion and Deletion Operations

Insertion and deletion operations involve not only the basic BST operations but also additional steps to maintain the red-black properties.

- **Insertion Complexity**: $O(\log n)$ for finding the correct position plus a constant number of rotations and color changes.
- **Deletion Complexity**: $O(\log n)$ for locating and removing the node, followed by a maximum of three rotations to fix violations.

Space Complexity Overview

The space complexity of a red-black tree is determined by the number of nodes and the overhead of storing color information.

- **Space Complexity**: $O(n)$, where n is the number of nodes. Each node requires space for its value, pointers to left and right children, a color bit, and a parent pointer.

Example of Complexity Analysis

Consider a red-black tree with n nodes:

```
    20B
   /    \
 10R    25R
 / \    /  \
5B 15B 22B 30B
```

- **Search Complexity**: To find any node, the maximum number of comparisons is the height of the tree, which is $O(\log n)$.
- **Insertion/Deletion Complexity**: Inserting or deleting a node requires traversing the tree down to a leaf and then performing rotations and color changes, all within $O(\log n)$ time.
- **Space Complexity**: The tree has n nodes, each requiring a fixed amount of space, resulting in $O(n)$ space complexity.

Conclusion

Red-black trees offer logarithmic time complexity for search, insertion, and deletion due to their self-balancing nature. The space complexity is linear with respect to the number of nodes. These properties make red-black trees an efficient choice for a balanced search tree, providing a good trade-off between time and space efficiency.

Formative Assessment 1

Formative assessment 2